Libro de Acceso

Control de Gastos para Estudiantes

Proyecto ACA — Programación Avanzada

Documento de referencia con términos, definiciones y guía de uso del programa

Problemática: El dinero del mes se acaba y el estudiante no sabe en qué se le fue, porque no anota los gastos pequeños del día a día.

Solución: Programa en Python que registra ingresos y gastos por categoría, muestra el saldo, un resumen de en qué se gasta más y alertas de presupuesto.

7 de noviembre de 2026

Índice general

1. Presentación **1**

2. Glosario de términos y definiciones **2**

3. Requisitos de acceso **4**

4. Estructura del programa **5**

4.1. Base compartida 5

4.2. Integrante 1 — Tickets 1, 2 y 3 5

4.3. Integrante 2 — Tickets 4, 5 y 6 5

4.4. Integrante 3 — Tickets 7, 8 y 9 6

4.5. Menú principal 6

5. Estructura de datos: el movimiento **7**

6. Guía de uso del menú principal **8**

7. Mensajes del sistema **10**

8. Ejemplos de flujos de uso **11**

9. Reglas de trabajo del equipo **12**

10. Preguntas frecuentes **13**

11. Índice de funciones **14**

12. Conclusión **15**

Capítulo 1

Presentación

Este **Libro de Acceso** reúne los términos, definiciones y procedimientos necesarios para usar, entender y mantener el programa **"Control de Gastos para Estudiantes"**. Está dirigido a:

- **Usuarios finales**: estudiantes que registran sus gastos.
- **Integrantes del equipo**: programan o revisan el código.
- **Docentes**: evalúan el proyecto.

Problemática y solución

La problemática que resuelve el programa es que *"el dinero del mes se acaba y el estudiante no sabe en qué se le fue"*, porque no anota los gastos pequeños del día a día. La solución es un programa en Python que registra ingresos y gastos por categoría, muestra el saldo, resume en qué se gasta más y genera alertas de presupuesto.

Capítulo 2

Glosario de términos y definiciones

Esta es la sección central del libro: **todos los términos clave** usados en el programa, ordenados alfabéticamente.

Término	Definición
Alerta de presupuesto	Aviso que muestra el programa cuando los gastos alcanzan el **80 %** (advertencia) o el **100 %** (alerta) del presupuesto mensual definido. Se genera con la función `verificar_presupuesto()`.
Base compartida	Conjunto de datos y funciones de apoyo (categorías, lista de movimientos, funciones de formato y validación) que **todos los integrantes usan** y que **no deben modificarse sin avisar al grupo**.
Categoría	Clasificación de un gasto. El programa define cinco: **Transporte, Comida, Fotocopias, Entretenimiento y Otros**. Se guardan en la constante `CATEGORIAS`.
Descripción	Texto corto que explica un movimiento (ej.: "Almuerzo", "Mesada", "Bus"). Es obligatorio y no puede quedar vacío.
Fecha	Día en que se registra un movimiento, en formato **DD/MM/AAAA** (ej.: `30/10/2026`). Se obtiene automáticamente con `fecha_hoy()`.
Gasto	Movimiento en el que **sale dinero**. Tiene `tipo = "gasto"`, una **categoría** obligatoria, descripción, valor y fecha.
Historial	Lista completa de todos los movimientos registrados (ingresos y gastos), mostrada en forma de tabla por `ver_historial()`.
Ingreso	Movimiento en el que **entra dinero** (mesada, trabajo, regalo, etc.). Tiene `tipo = ïngreso"` y su categoría es `None` (no aplica).
Menú principal	Interfaz de consola con 9 opciones numeradas (1–9) más la opción **0. Salir**. Es el punto de entrada del programa (`menu()`).
Movimiento	Registro individual de un ingreso o un gasto. Es un **diccionario** con las claves `tipo`, `categoria`, `descripcion`, `valor` y `fecha`.
Pendiente	Estado de una función que aún no ha sido programada por su responsable. Muestra el mensaje `Función pendiente (Ticket N - Integrante)` mediante `pendiente()`.

Término	**Definición**
Pesos colombianos	Moneda en la que se expresan los valores. Se formatean con `formato_pesos()` (ej.: `12500` → `$12.500`).
Presupuesto mensual	Límite máximo de gasto que el estudiante se fija para el mes. Se guarda en la variable global `presupuesto`. Si es `0`, significa **"sin definir"**.
Saldo	Dinero disponible = **total de ingresos − total de gastos**. Se calcula con `calcular_totales()`. Puede ser negativo.
Ticket	Tarea asignada a un integrante del equipo (por ejemplo, "Ticket 4: registrar gasto"). Cada ticket corresponde a una o más funciones.
Validación	Verificación de que un dato ingresado sea correcto. Ej.: `pedir_valor()` exige números mayores que cero; `pedir_texto()` exige texto no vacío.
Valor	Cantidad de dinero de un movimiento, en pesos y **mayor que cero**. Se captura con `pedir_valor()`.

Capítulo 3

Requisitos de acceso

Para ejecutar el programa se necesita:

- **Python 3.7 o superior** (usa *f-strings* y `calendar.monthrange`).
- Un editor o IDE (VS Code, PyCharm, IDLE) **o** una terminal.
- **No requiere librerías externas**: solo usa `calendar` y `datetime`, incluidas en Python.

Cómo ejecutar:

```
python nombre_del_archivo.py
```

Capítulo 4

Estructura del programa

El archivo está organizado en **cuatro bloques**.

4.1. Base compartida

Contiene lo que **todos** usan:

Elemento	Tipo	Función
`CATEGORIAS`	Lista	Las 5 categorías de gasto.
`movimientos`	Lista	Almacena todos los movimientos (diccionarios).
`presupuesto`	Entero	Presupuesto mensual (0 = sin definir).
`formato_pesos()`	Función	Convierte número → `$12.500`.
`pedir_valor()`	Función	Pide un valor > 0 y valida.
`pedir_texto()`	Función	Pide texto no vacío.
`fecha_hoy()`	Función	Devuelve la fecha actual DD/MM/AAAA.
`calcular_totales()`	Función	Devuelve (ingresos, gastos, saldo).
`pendiente()`	Función	Marca funciones no terminadas.

4.2. Integrante 1 — Tickets 1, 2 y 3

- `registrar_ingreso()` — Ticket 1
- `ver_historial()` — Ticket 2
- `definir_presupuesto()` / `verificar_presupuesto()` — Ticket 3

4.3. Integrante 2 — Tickets 4, 5 y 6

- `registrar_gasto()` — Ticket 4
- `ver_saldo()` — Ticket 5
- `eliminar_ultimo()` — Ticket 6

4.4. Integrante 3 — Tickets 7, 8 y 9

- `ver_resumen()` — Ticket 7
- `gasto_diario_disponible()` — Ticket 8
- `buscar_por_categoria()` — Ticket 9

4.5. Menú principal

`menu()` integra todas las opciones y es lo que se ejecuta al iniciar el programa.

Capítulo 5

Estructura de datos: el movimiento

Cada movimiento es un **diccionario** con esta forma exacta:

```
{
    "tipo": "ingreso" | "gasto",
    "categoria": "Comida" | None,    # None si es ingreso
    "descripcion": "Almuerzo",
    "valor": 12000,                  # entero > 0
    "fecha": "30/10/2026"            # DD/MM/AAAA
}
```

Ejemplo de ingreso:

```
{"tipo": "ingreso", "categoria": None, "descripcion": "Mesada",
 "valor": 200000, "fecha": "01/10/2026"}
```

Ejemplo de gasto:

```
{"tipo": "gasto", "categoria": "Transporte", "descripcion": "Bus",
 "valor": 2500, "fecha": "02/10/2026"}
```

Importante: todos los integrantes deben respetar esta estructura, porque las funciones de resumen, historial y búsqueda la usan directamente.

Capítulo 6

Guía de uso del menú principal

Al ejecutar el programa aparece el menú principal con las siguientes opciones.

Opción 1 — Registrar ingreso

Pide **descripción** y **valor**. Guarda un movimiento `tipo = ïngreso"` con la fecha del día.

Ejemplo: `Ingreso registrado: Mesada por $200.000`

Opción 2 — Registrar gasto

1. Muestra las **5 categorías numeradas**.
2. El usuario elige una (número válido).
3. Pide **descripción** y **valor**.
4. Guarda el gasto y, si el **saldo queda negativo**, muestra `¡Cuidado! Su saldo quedó negativo`.
5. El menú llama automáticamente a `verificar_presupuesto()`.

Opción 3 — Ver saldo

Muestra **total ingresos**, **total gastos** y **saldo disponible**. Si hay presupuesto, agrega el **presupuesto mensual** y el **restante**.

Opción 4 — Resumen por categoría

Calcula cuánto se gastó en cada categoría y su **porcentaje** sobre el total de gastos. Indica **la categoría en la que más se gasta**.

Opción 5 — Historial de movimientos

Muestra una **tabla** con columnas: N°, Fecha, Tipo, Categoría, Descripción, Valor. Al final, los totales y el saldo.

Opción 6 — Buscar gastos por categoría

Pide elegir una categoría y lista **solo los gastos** de esa categoría, con el **total** al final.

Opción 7 — ¿Cuánto puedo gastar por día?

Con base en el saldo actual y los **días restantes del mes**, calcula un **gasto diario aproximado**. Si el saldo es ≤ 0, avisa que no es posible calcularlo.

Opción 8 — Definir presupuesto mensual

Pide el presupuesto (valor > 0), lo guarda en la variable global y ejecuta una verificación inmediata de alertas.

Opción 9 — Eliminar último movimiento

Muestra los datos del **último movimiento** y pide confirmación (s/n). Solo elimina si el usuario responde **"s"**.

Opción 0 — Salir

Termina el programa con un mensaje de despedida.

Capítulo 7

Mensajes del sistema

Símbolo	Significado
✔	Operación completada con éxito.
⚠	Advertencia (dato inválido, gasto al 80 %, etc.).
🔔	Alerta grave (saldo negativo o presupuesto superado).
⏳	Función pendiente por programar.
—	Categoría no aplica (en ingresos) o vacía.

Capítulo 8

Ejemplos de flujos de uso

Flujo A: registrar mesada y primer gasto

```
1 -> Descripcion: Mesada -> Valor: 200000
2 -> Categoria: 2 (Comida) -> Descripcion: Almuerzo -> Valor: 12000
3 -> Ver saldo -> Saldo: $188.000
```

Flujo B: controlar el presupuesto

```
8 -> Presupuesto: 300000
2 -> Categoria: 1 (Transporte) -> Valor: 250000
   Cuidado: ya uso el 83 % del presupuesto. Le quedan $50.000.
3 -> Ver saldo -> Restante del presupuesto: $38.000
```

Flujo C: analizar los gastos

```
4 -> Resumen: Comida 40 %, Transporte 35 %, ...
6 -> Buscar: 2 (Comida) -> Lista de gastos + total
7 -> Cuanto puedo gastar por dia? -> $X diarios
```

Capítulo 9

Reglas de trabajo del equipo

1. **No modificar la base compartida** sin avisar al grupo.
2. Cada integrante completa **solo las funciones de sus tickets**.
3. Mientras una función no esté lista, debe mostrar `pendiente(...)` para que el programa **siga ejecutándose**.
4. Respetar la **estructura del diccionario de movimiento**.
5. Usar `pedir_valor()` y `pedir_texto()` para validar entradas.
6. Usar `formato_pesos()` para mostrar dinero.
7. Probar siempre desde el **menú principal** (ejecutable de punta a punta).

Capítulo 10

Preguntas frecuentes

¿Qué pasa si escribo letras en un valor?
`pedir_valor()` lo detecta y repite la pregunta: `Escriba solo números, sin letras. Ej: 15000`.

¿Puedo registrar un gasto sin categoría?
No. La opción 2 obliga a elegir un número del 1 al 5.

¿Qué ocurre si el saldo es negativo?
El programa lo advierte con una alerta, pero **no impide** registrar el gasto.

¿Cómo borro un error?
Usa la opción **9** (eliminar último movimiento) y confirma con **"s"**. Solo elimina el último, no cualquiera.

¿Dónde se guardan los datos?
En la lista `movimientos` **en memoria**. Al cerrar el programa se pierden (no hay persistencia en archivo en esta versión).

¿Por qué aparece "Función pendiente"?
Porque algún ticket aún no ha sido programado por su responsable. El programa sigue funcionando para no bloquear el trabajo del resto.

Capítulo 11

Índice de funciones

Función	Ticket	Responsable	Propósito
`registrar_ingreso`	1	Integrante 1	Guardar un ingreso.
`ver_historial`	2	Integrante 1	Tabla de movimientos.
`definir_presupuesto`	3A	Integrante 1	Fijar presupuesto.
`verificar_presupuesto`	3B	Integrante 1	Alertas 80 % / 100 %.
`registrar_gasto`	4	Integrante 2	Guardar un gasto.
`ver_saldo`	5	Integrante 2	Ingresos, gastos, saldo.
`eliminar_ultimo`	6	Integrante 2	Borrar último movimiento.
`ver_resumen`	7	Integrante 3	Gastos por categoría + %.
`gasto_diario_disponible`	8	Integrante 3	Gasto diario sugerido.
`buscar_por_categoria`	9	Integrante 3	Filtrar y sumar por categoría.
`menu`	—	Base compartida	Menú y control general.

Capítulo 12

Conclusión

Este **Libro de Acceso** concentra los **términos, definiciones, estructura y procedimientos** del programa. Sirve como:

- **Manual de usuario** para el estudiante que registra sus gastos.
- **Documentación técnica** para el equipo de desarrollo.
- **Guía de referencia** para evaluar el cumplimiento de los tickets.

Recomendación: mantener este documento actualizado cada vez que se agregue una nueva función, categoría o mensaje al programa, y compartirlo con todos los integrantes antes de cada entrega.